Asad Farooq | 247171
Usama Arshad | 247141
Ibtehaj Ali Mirza | 247200

Computer Organization and Assembly Language

Dr. Sonia Shahzadi

Air University Kharian

Project

December 29, 2025

Technical Report: Binary/Hexadecimal Memory Inspection Utility

Subject: Implementation of a Low-Level Memory Viewer in 8086 Assembly

Developer Environment: emu8086 / DOS Real Mode

I. **Introduction**

This project is based on 8086 Assembly Language and is developed to view memory contents in hexadecimal and binary formats. The main idea of this project is to help understand how data is stored in memory and how it can be displayed in different number systems using low-level programming.

The program takes input from the user, such as a memory offset, number of bytes, and display mode. According to the selected mode, the memory contents are shown in hex, binary, or both. This project was implemented in the EMU8086 / DOS environment and uses DOS interrupts for input and output operations.

II. **Purpose of the Project**

The purpose of this project is:

i. To understand how memory addressing works in 8086.
ii. To practice assembly language programming.
iii. To learn how to use DOS interrupts (INT 21h).
iv. To display data in hexadecimal and binary formats.
v. To apply input validation and error handling.
vi. To improve understanding of bitwise and arithmetic operations.

III. **Tools and Environment**

The following tools were used in this project:

- EMU8086 as the assembler and emulator
- 8086 processor architecture
- DOS interrupt services
- Small memory model

IV. **How the Program works**

When the program starts, it first displays the project title. After that, it asks the user to enter:

i. A starting memory offset in hexadecimal.
ii. The number of bytes to display (also in hexadecimal).
iii. The display mode:

- H for Hexadecimal
- B for Binary
- A for both Hex and Binary

If the user enters any invalid value, the program shows an error message and asks for input again. Once valid input is provided, the program reads memory from the given offset and displays the output in a proper format. Finally, the program waits for a key press and exits.

V. **Data Segment Description**

The data segment contains:

i. Messages for user prompts, errors, and output headers.
ii. Input buffers for offset, length, and mode using DOS buffered input.
iii. Variables to store:

- Starting offset
- Length of memory
- Display mode

iv. Formatting strings like newline, space, and colon.
v. A small demo memory area for testing and display purposes.

VI. **Main Procedure Explanation**

The main procedure controls the complete flow of the program.

First, the data segment is initialized. Then the program prints the title on the screen. After that, user input is taken step by step. The offset and length are entered in hexadecimal format and converted into numeric values.

The program checks whether the entered length is between 0001h and 00FFh. It also checks whether the selected mode is valid. If any input is wrong, an error message is shown and the user is asked to enter the values again.

Once everything is correct, the memory dump procedure is called to display the required output.

VII. **Memory Dump Logic**

The DumpMemory procedure is the core part of this project.

It starts reading memory from the given offset using the SI register. The number of bytes to display is controlled by the CX register. For better readability, the program prints the memory offset at the start of every 16 bytes.

Depending on the selected mode:

- Only hexadecimal values are printed.
- Only binary values are printed.
- Or both hexadecimal and binary values are printed.

Proper spacing and formatting are used to make the output clear and easy to read.

VIII. **Hexadecimal Input Conversion**

The hexadecimal input entered by the user is converted into numeric values using a separate procedure. Each character is checked to make sure it is a valid hex digit. The value is built by multiplying the previous result by 16 and adding the new digit.

If an invalid character is found, the program immediately shows an error message. This helps prevent wrong memory access.

IX. **Hexadecimal and Binary Output**

For hexadecimal output, each byte is split into two nibbles (upper and lower). These nibbles are converted into their corresponding ASCII characters and displayed on the screen.

For binary output, each bit of the byte is checked one by one using bitwise operations. The program prints 0 or 1 accordingly. This helps in understanding how data is actually stored at the bit level.

Special care is taken to preserve register values, which fixes common issues that occur in assembly programs.

X. **DOS Interrupts Used**

The program uses several DOS interrupts for input and output:

- Printing strings on the screen
- Printing single characters
- Taking buffered input from the keyboard
- Waiting for a key press
- Exiting the program properly

These interrupts make the program interactive and user-friendly.

XI. **Error Handling**

The program includes proper error handling. It checks for:

- Invalid hexadecimal input
- Empty input
- Length outside the allowed range
- Invalid display mode

If any error occurs, the user is informed and asked to re-enter the data. This makes the program more reliable.

XII. **Assembly Concepts Used**

This project demonstrates several important assembly language concepts:

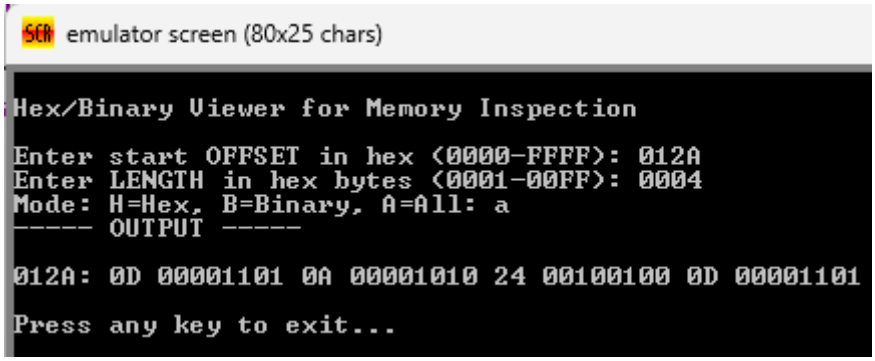
- Procedures and modular programming
- Stack usage and register preservation
- DOS interrupts
- Bitwise operations
- Arithmetic operations
- String and buffer handling

XIII. **Conclusion**

In conclusion, this project helped in understanding how memory can be accessed and displayed using 8086 Assembly Language. It also improved skills in handling user input, formatting output, and applying error checking. The project is useful for learning low-level programming and understanding how data is represented in different formats at the hardware level.

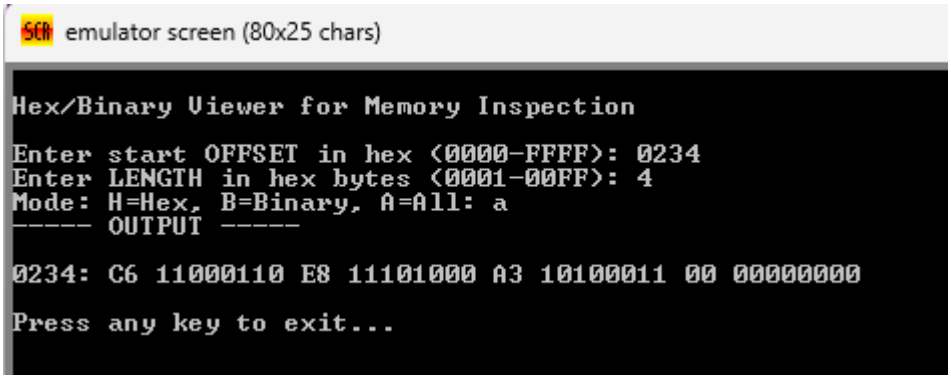
XIV. **Sample Outputs**

1.



- **Memory Addressing:** The output begins at the start offset 012A, which serves as the base memory address for the inspection.
- **Data Quantity:** The tool displays exactly 4 bytes of data, corresponding to the "LENGTH in hex bytes" input of 0004.
- **Dual Representation:** Since the mode selected was 'a' (All), each byte is presented in two formats: its two-digit Hexadecimal value followed by its 8-bit Binary equivalent.
- **Value Mapping:** The data found at this location consists of the hex values 0D, 0A, 24, and 0D, which in technical contexts often represent control characters (like Carriage Return and Line Feed) and the '$' symbol.

2.



- **Memory Addressing:** The inspection starts at the specific memory address entered by the user, such as 012A or 0234, which is displayed at the beginning of the output line.
- **Data Volume:** The output displays a set number of bytes—specifically 4 bytes in these examples—based on the "LENGTH in hex bytes" input provided.
- **Multi-Format Display:** Since the mode is set to 'a' (All), the tool provides a side-by-side view of each byte, showing its Hexadecimal value followed immediately by its 8-bit Binary equivalent.
- **Raw Data Inspection:** The values represent the actual contents of the system's memory at that moment; for instance, the sequence 0D 0A 24 0D corresponds to control characters and symbols used in text formatting.

THE END